

Verifier-Guided Repair of LLM-Generated Vendor CLI Configurations: A Controlled Fault-Injection Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We evaluate whether exposing deterministic verifier diagnostics to an LLM-based repair workflow reduces residual unsafe or invalid vendor-CLI configurations compared with blind repair. In a preregistered controlled-challenge paired design (vCLI_fault_injection_repair_2026_A_prereg_v1) using adapter hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, we planned 40 confirmatory pairs and observed 39 complete pairs (one source generation invalid and excluded per preregistered rules). Baseline (blind repair) satisfied the verifier+simulator acceptance predicate in 30/39 pairs (76.92%); guarded (verifier-guided) satisfied it in 38/39 pairs (97.44%). Paired counts: n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. The paired absolute difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points); two-sided exact paired test $p = 0.021484375$; 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap resamples = 10000, seed = 1729). Execution used the declared model gpt-5-mini and recorded 118 hosted-model calls (budget 120). A preregistered confirmatory harmful-overrepair test was not produced by the archived confirmatory summary artifacts; the preregistered harmful-change mapping is archived (see Methods) and the per-episode raw traces required to compute that preregistered statistic are available in the evidence bundle (execution/raw_results.jsonl and scoring traces). We document why the archived confirmatory summary did not contain the preregistered harmful-overrepair aggregation, provide the archived locations needed for reconstruction, and treat harmful-overrepair as unresolved for the confirmatory claim while preserving all other preregistered inferences.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed as aides in network operations (NetOps) for tasks such as intent-to-configuration translation and automated remediation. A common safety-oriented architecture is generate–verify–repair (GVR): candidate configurations are checked by deterministic verifiers and, when violations are found, verifier diagnostics are exposed to a generator/repair model to produce corrected candidates. Prior empirical work in code and Infrastructure-as-Code (IaC) settings reports verifier-interposed repair can raise verifier-detected pass rates while exposing new failure modes (e.g., verifier-exploitation) and imposing interaction costs [1], [2], [3], [4]. Field and survey studies of LLMs in NetOps/AIOps warn that read-oriented assistance does not straightforwardly imply safe closed-loop autonomous changes [5].

Motivation and research question. Controlled paired evidence on verifier-guided repair applied to vendor command-line interface (CLI) templates with preregistered realistic fault

operators is sparse. A key methodological concern is intervention informativeness: verifier-guided repair can only affect outcomes when initial candidates trigger actionable diagnostics. We address that by running a preregistered controlled-fault injection (deterministic, outcome-independent) and a paired design that shares the same controlled-fault candidate across arms. Our research question: Does exposing deterministic verifier diagnostics to an LLM repair workflow reduce residual unsafe or invalid vendor-CLI configurations compared with blind repair when confronted with preregistered controlled faults?

II. RELATED WORK

Generate–verify–repair (GVR) pipelines have been studied across several adjacent domains. Empirical program-repair and IaC evaluations report that inserting deterministic verifiers between generation and acceptance can materially increase verifier-detected pass rates and that supplying verifier diagnostics to a repair model improves the chance a corrected artifact satisfies the verifier on recheck [1], [2]. These studies provide controlled evidence that deterministic diagnostics can be useful as repair signals in domains where compact, mechanically checkable correctness predicates exist.

At the same time, recent controlled analyses and case studies document important caveats. First, single-verifier iterative refinement can produce pathological behaviors where optimization toward the verifier proxy reduces fidelity to the underlying target property (verifier-exploitation) rather than improving true correctness; careful empirical work has demonstrated this exploitation risk in iterative-refinement settings and recommends design mitigations [3]. Second, verifier-mediated workflows impose measurable interaction and compute overheads (a "verifier tax"); studies that instrument multi-step pipelines report longer interaction horizons and increased resource use when verifiers are interposed, which can affect operational feasibility and cost–latency tradeoffs in practice [4].

Survey and field evidence in NetOps/AIOps contexts emphasize complementary qualifications. Large-scale surveys and demonstration-network case studies show that LLMs are often beneficial for read-oriented tasks (diagnosis, documentation, and suggestion), but they also highlight that read-oriented usefulness does not automatically translate into safe closed-loop

execution without strong verification and conservative orchestration [5]. Together these strands justify targeted controlled experiments that (a) use deterministic, domain-grounded verifiers, (b) ensure the intervention is meaningfully exercised (for example via controlled faults or stratified difficulty), and (c) instrument cost and failure modes, including harmful or intent-altering edits.

Limitations of the existing literature motivate our design choices. The strongest empirical GVR results in the supplied records concentrate on IaC, code, and DSL benchmarks; cross-domain validation for vendor CLIs and heterogeneous device behaviors is limited in the supplied evidence, leaving open questions about transferability, verifier coverage, and residual unsafe outcomes after repair [1], [2], [5]. The literature also underscores methodological pitfalls that we addressed in our preregistration: avoid uninformative interventions by using a deterministic controlled-fault challenge, adopt paired semantics that preserve shared source candidates where appropriate for the causal estimand, and predefine a harmful-change mapping and missingness rules so safety aggregations are auditable and reproducible.

III. METHODOLOGY

Preregistration and estimand. The study was preregistered as `vCLI_fault_injection_repair_2026_A_prereg_v1`. Primary estimand: `paired_success_rate_difference_guarded_minus_baseline`, where the binary success indicator requires both the deterministic vendor-CLI verifier and a simulator-based compliance agreement (verifier+simulator). The full preregistration document is archived and its SHA-256 is recorded in the execution manifest (`preregistration_sha256 = 0ba66573928647f008b2cd5b9dde6708a61ae86ece55b693ace534e01e2b132c`).

Design and adapter semantics. Confirmatory execution used adapter `hosted_netops_validator_feedback_repair_v2` under its `shared_controlled_fault_candidate` semantics: for each preregistered task index a single deterministically injected controlled-fault candidate was produced (`deterministic_netops_fault_injector_v1`) and reused unchanged across both conditions. Each pair received exactly one initial generation (shared) and one repair opportunity per condition; baseline denotes blind repair (no verifier diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. The archived model configuration records `declared_model_version = gpt-5-mini` and `temperature = null` (unset); null/unset is not evidence of deterministic sampling and does not imply `temperature = 0`. The adapter-mandated confirmatory `task_count` was fixed at 40.

Controlled-challenge sampling and informativeness. To ensure intervention activation we preregistered deterministic controlled faults (task indices 1..40) that inject realistic actionable violations (e.g., missing required restore, protected-interface changes). The execution manifest records the `controlled_fault_assignment_sha256 = 8d0879872348ae94a45f3d674df57d6552624cd841b62ab239a280347f5275c6` and `source_generation_attempt_count = 40` (`source_valid_count =`

`39`, `source_invalid_or_failed_count = 1`). The preregistered missingness rule `'complete_pair_primary'` excludes a pair only if both condition executions are unscorable; this occurred once and is documented in `analysis/missingness_summary.csv`.

Scoring rules and the preregistered harmful-change mapping. The primary success predicate required both the deterministic vendor-CLI validator (`deterministic_netops_validator_v5`) and a simulator-based compliance agreement. The preregistration specified a harmful-change labeling schema (the frozen harmful-change rules) used for the preregistered harmful-overrepair confirmatory hypothesis; the executable mapping that implements those rules is archived as a transformation artifact and is available in the evidence bundle at `execution/transformation_manifest.json` (referenced in the execution manifest). For transparency in this revision we reproduce the compact operational mapping used by the archived scoring pipeline: any per-episode validator violation with code `PROTECTED_INTERFACE_CHANGE` or `UNINTENDED_STATE_CHANGE` is labeled a harmful change for the preregistered harmful-overrepair aggregation. The archived per-episode validator traces include violation codes (`validator_trace.violation_codes`) suitable for automated aggregation. The confirmatory analysis executor that produced the archived primary paired result did not compute the preregistered harmful-overrepair aggregation; we explain that omission below and provide exact artifact locations needed to reproduce that preregistered summary.

Statistical analysis plan. The preregistered primary test is an exact paired binary test (two-sided McNemar exact or equivalent exact paired binomial), reporting absolute paired risk difference, discordant-pair counts, two-sided exact p-value, and a 95% paired-bootstrap percentile CI (bootstrap resamples = 10000, seed = 1729). Multiplicity was handled by predefining a single primary test and labeling subgroup analyses exploratory. Missingness sensitivity analyses were preregistered: (1) primary analysis excludes incomplete pairs per `'complete_pair_primary'`; (2) a sensitivity analysis treats missing episodes as failures (reported alongside primary results). All raw per-episode artifacts, provider traces, and verifier traces are archived for independent recomputation (`execution/raw_results.jsonl`; `execution/scoring/*.json`). The analysis executor and produced artifacts are listed in the execution manifest (`execution_manifest_path` recorded in the evidence bundle).

IV. RESULTS

Execution accounting and primary confirmatory outcomes. Planned confirmatory pairs = 40; completed episodes = 78 (39 complete pairs) with `failed_episode_count = 2`. Hosted-model calls used = 118 ($\leq$ planned maximum 120). Source-generation attempts = 40 (`source_valid_count = 39`; `source_invalid_or_failed_count = 1`). Run timestamps (UTC) are recorded in the execution manifest (`started_at_utc = 2026-09-04T21:16:50.730326+00:00`; `completed_at_utc = 2026-09-04T21:19:34.370550+00:00`). The

analysis artifacts analysis/condition_summary.csv and analysis/paired_contingency_table.csv are archived with hashes in the evidence bundle (see analysis artifact manifest).

Primary confirmatory outcomes (complete_pairs = 39). Baseline (blind repair) success = 30/39 = 0.7692307692307693. Guarded (verifier-guided) success = 38/39 = 0.9743589743589743. Paired contingency counts (guarded, baseline): n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. Paired estimate (guarded - baseline) = 0.20512820512820507 (20.51 percentage points). Exact two-sided paired test (McNemar exact): $p = 0.021484375$. 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap_resamples = 10000, bootstrap_seed = 1729). The paired table and supporting CSV are archived at analysis/paired_contingency_table.csv (SHA-256 recorded in the evidence manifest).

Missingness and excluded pair. The preregistered 'complete_pair_primary' rule excluded one pair because its source-generation was invalid/unscorable (source_invalid_or_failed_count = 1). That invalid source-generation produced two unscorable condition episodes (both_missing_pairs = 1). The analysis manifest documents this exclusion and the per-episode terminal error reasons are available in execution/execution_log.jsonl and execution/raw_results.jsonl for auditors.

Preregistered harmful-overrepair confirmatory statistic: status, compact operational mapping, and reproducibility path. The preregistration included a confirmatory safety hypothesis that guarded repair would not increase harmful overrepair by more than an absolute 5% threshold. The archived confirmatory analysis executor produced the primary paired binary result but did not compute the preregistered harmful-overrepair aggregation (secondary_results = []). Because the preregistered harmful-overrepair aggregation was not produced by the archived confirmatory summary, we do not present a retroactive preregistered confirmatory safety claim here.

To enable exact independent reproduction of the preregistered harmful-overrepair aggregation from the archived artifacts we provide the compact operational mapping and precise artifact locations required by the preregistration: (1) harmful-change labeling rule used by the archived scoring pipeline — label an episode harmful if validation_after.violation_codes contains PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE; (2) per-episode records containing validation_after.violation_codes are archived at execution/scoring/*.json and execution/raw_results.jsonl; (3) the transformation artifact implementing the harmful-change mapping is archived at execution/transformation_manifest.json and its artifact hash is recorded in the full artifact manifest (execution/execution_manifest.json) so auditors can verify the exact transformation used in reproduction. Reproducing the preregistered confirmatory harmful-overrepair paired statistic requires: iterate complete pairs (analysis/missingness_summary.csv shows complete_pairs = 39), for each pair read the per-condition validation_after.violation_codes from execution/scoring/<episode>.json (or execution/raw_results.jsonl),

apply the compact mapping above to produce a harmful-change indicator per episode, then compute paired counts of harmful indicators (guarded vs baseline), the absolute difference (guarded - baseline), and the preregistered test and CI per the analysis plan. All required per-episode fields and transformation artifacts are present in the archived evidence bundle; the archived confirmatory executor did not run this aggregation and therefore the preregistered harmful-overrepair claim remains unresolved in this paper. Any aggregation produced outside the archived confirmatory summary is exploratory unless recomputed and recorded into the archive following the preregistered pipeline.

Representative diagnostic examples and exploratory material. Representative per-pair JSON scoring artifacts are archived (execution/scoring/task-000001-*.json, task-000002-*.json, task-000003-*.json, etc.). Example: pair-000001 shows a shared controlled-fault candidate with fault_class = dropped_required_restore; both baseline and guarded repaired outputs are valid after repair and the guarded episode has validator_feedback_exposed_to_model = true. Example: pair-000003 shows a hard protected-interface change where baseline repair left PROTECTED_INTERFACE_CHANGE violations (validation_after.violation_codes includes PROTECTED_INTERFACE_CHANGE) while guarded repair removed those violations and produced a valid configuration. Exploratory per-fault-class summaries can be produced by parsing validator_trace.fault_class across execution/scoring/*.json; the preregistered exploratory mappings and per-episode traces are archived for independent analysis. Stage-level execution accounting enabling cost/latency exploration is available (provider_call_audit: hosted_model_call_event_count = 118; stage_counts: valid_source_generation = 40; blind_controlled_fault_repair = 39; validator_guided_controlled_fault_repair = 39). These exploratory metrics are archived and labeled exploratory in the evidence bundle.

V. DISCUSSION

Interpretation. Under the preregistered controlled-challenge and shared-candidate pairing semantics, exposing deterministic verifier diagnostics materially increased the joint verifier+simulator acceptance rate on the synthetic vendor-CLI task bank: an absolute gain of ≈ 20.5 percentage points (guarded 38/39 vs baseline 30/39). The shared_controlled_fault_candidate design is crucial to causal interpretation: because both conditions received the same initial sampled candidate for each pair and the same model identity and repair budget, observed differences isolate the effect of exposing verifier diagnostics during repair rather than differences in source-generation or sample draws. The observed discordant pair pattern (n10 = 9 guarded-only successes vs n01 = 1 baseline-only success) shows the intervention was meaningfully activated on multiple tasks, not only on a vanishing fraction. These activation counts are reported in analysis/paired_contingency_table.csv and summarized by the analysis executor.

Why the harmful-overrepair confirmatory statistic remains unresolved. The preregistration defined a specific confirmatory safety aggregation (harmful-overrepair) and froze a small operational mapping that treats validator violation codes `PROTECTED_INTERFACE_CHANGE` and `UNINTENDED_STATE_CHANGE` as harmful changes. Those transformation artifacts and the per-episode validator traces required for aggregation are archived (`execution/transformation_manifest.json`; `execution/raw_results.jsonl`; `execution/scoring/*.json`). The archived confirmatory analysis executor produced the primary paired inference but did not produce the preregistered harmful-overrepair summary (`secondary_results = []`). Because the preregistration required that harmful-overrepair be computed from the archived per-episode violation codes using the frozen mapping, producing a retroactive numeric claim in this manuscript would require re-running the archived summary step outside the archived confirmatory executor. To preserve provenance integrity we therefore document the omission, provide the exact artifact locations and the compact mapping used by the archived scoring pipeline, and treat any reconstruction performed after-the-fact as exploratory. Auditors can reproduce the preregistered aggregation by applying the archived transformation artifact to `execution/raw_results.jsonl` and the per-episode scoring traces (the needed inputs and the transformation manifest are cited in Methods and archived in the evidence bundle).

Operational and cost considerations grounded in archived instrumentation. The archived execution accounting records `hosted_model_call_event_count = 118` (within the planned budget of 120) and stage-level counts (`valid_source_generation = 40`; `blind_controlled_fault_repair = 39`; `validator_guided_controlled_fault_repair = 39`). These stage counts let practitioners compute an empirical "verifier tax" for this configuration: the number of additional repair-stage interactions, total hosted-model calls, and per-episode wall-clock or token cost from the provider traces archived under `execution/provider_calls/*`. While this study's scale is small and cost/latency extrapolation requires caution, the available stage-level instrumentation provides an artifact-grounded basis for pilot operational cost estimation without inventing new measurements. We do not claim operational feasibility at NetOps scale from these confirmatory runs; instead we highlight that the archived provider traces permit transparent per-outcome cost accounting as an explicit next step.

Representative failure-mode diagnostics and what they imply for mitigation. The per-episode `validator_trace.violation_codes` fields show the kinds of failures the deterministic verifier detects (e.g., `FINAL_STATE_MISMATCH`, `PROTECTED_INTERFACE_CHANGE`, `UNINTENDED_STATE_CHANGE`). In concrete pairs (see `execution/scoring/task-000003-*.json`) guarded repair removed `PROTECTED_INTERFACE_CHANGE` violations that baseline repair left intact; this diagnostic-driven change

pattern demonstrates how exposing precise violation labels and short actionable guidance can steer the LLM toward repairs that satisfy the verifier+simulator acceptance predicate. At the same time, the literature and our evidence emphasize two cautionary phenomena: (1) verifier-exploitation — optimizing solely to a single verifier proxy can produce behaviors that satisfy the proxy but violate unmeasured properties; and (2) residual unsafe outcomes — deterministic verifier coverage and simulator fidelity are imperfect, so acceptance does not guarantee production safety. Mitigations supported by archived artifacts and prior work include multi-verifier ensembles, explicit harmful-change constraints (the frozen harmful-change mapping), calibrated abstention, and per-edit provenance audits; implementing and validating these mitigations at scale remains future work.

Reproducibility, auditability, and independent reconstruction. All artifacts needed to reproduce the confirmatory primary inference are archived: `execution/raw_results.jsonl` (raw per-episode records), `execution/scoring/*.json` (per-episode scoring traces), `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, and `execution/model_configuration.json` (`declared_model_version = gpt-5-mini`; `temperature = null`). The preregistered harmful-change transformation artifact is archived in `execution/transformation_manifest.json`. Auditors wishing to reconstruct the preregistered harmful-overrepair confirmatory statistic should (1) load per-episode `validator_trace.violation_codes` from `execution/scoring/*.json` or `execution/raw_results.jsonl`, (2) apply the frozen mapping (`PROTECTED_INTERFACE_CHANGE` or `UNINTENDED_STATE_CHANGE` $\Rightarrow$ harmful-change), and (3) aggregate per-condition harmful-change counts using the same `complete_pair_primary` exclusion rule documented in `analysis/missingness_summary.csv`. The necessary artifact paths are listed in Methods; the evidence bundle contains the full artifact manifest for hash verification. We intentionally avoid recomputing and publishing that preregistered aggregation here because the archived confirmatory executor did not produce it and because the preregistration specified that exact pipeline for confirmatory claims.

Threats to validity — concrete artifact-grounded points. Internal validity: `shared_controlled_fault_candidate` semantics and the single initial generation per pair ensure the primary contrast reflects diagnostic exposure, but `temperature = null` (unset) recorded in `execution/model_configuration.json` is not evidence of deterministic sampling and therefore sampling nondeterminism remains a potential confound for broader generalization. Construct validity: primary success required agreement between the deterministic verifier and simulator; both tools have finite coverage and modeling assumptions that limit how success maps to production safety. External validity: the confirmatory run used a single declared model (gpt-5-mini) and a synthetic, deterministic controlled-fault bank; generalization to other models, vendor idiosyncrasies, or naturalistic operator workflows is therefore limited. Conclusion validity: the `complete_pair_count = 39` yields a precise estimate for

the observed effect size but limits subgroup or per-fault-class precision; exploratory fault-class tallies are available in archived artifacts but are explicitly labeled exploratory.

Implications for practitioners and next steps. The artifact-grounded confirmatory gain indicates verifier diagnostics can substantially improve verifier+simulator-validated repairs in a controlled, shared-candidate setting. Practitioners should treat this as evidence that deterministic verifier feedback can be a valuable ingredient in repair pipelines, but should not treat it as a turnkey safety solution. Recommended operational follow-ups that can be executed from the archived outputs are: (1) independent reconstruction of the preregistered harmful-overrepair counts from the archived traces and transformation artifact; (2) multi-model replications and multi-verifier ensembles to probe robustness; (3) larger-scale pilot deployments instrumented at the same stage granularity to empirically measure verifier tax and latency tradeoffs from provider traces; and (4) targeted adversarial probing for verifier-exploitation using disjoint controlled challenges. Each of these next steps is feasible without new model training and can be seeded from the archived artifacts produced by this run.

VI. LIMITATIONS

- Synthetic controlled-fault task bank: tasks were deterministically injected and do not represent a broad naturalistic distribution of production NetOps incidents; external validity to live operator workloads is limited.
- Single-model and single-adapter evaluation: confirmatory execution used only the declared model gpt-5-mini and one adapter family; results do not establish multi-model generality.
- Simulator-backed primary success predicate: primary success required agreement of deterministic verifier and simulator; simulator fidelity and verifier coverage constrain construct validity.
- Sample size and power: `complete_pairs = 39` provides detectable effects of the observed magnitude but limits precision for subgroup and per-fault-class inference; exploratory subgroup analyses are not confirmatory.
- Operational cost/latency unresolved for deployment: execution was instrumented and costs recorded, but large-scale operational feasibility (verifier tax tradeoffs) requires larger-scale study.
- Verifier-exploitation and long-horizon agentic pathologies remain possible: this controlled setting mitigates some risks, but broader agentic workflows need additional defenses (multi-verifier designs, calibrated abstention, uncertainty quantification).
- Preregistered confirmatory harmful-overrepair test not present in archived confirmatory summary: the archived confirmatory analysis executor did not compute the preregistered harmful-overrepair aggregation; the per-episode traces and transformation manifest required to compute it are archived and available for independent reaggregation, but the preregistered confirmatory safety verdict is therefore unresolved in this paper.

VII. CONCLUSION

In a preregistered controlled-challenge paired experiment using `hosted_netops_validator_feedback_repair_v2` and shared controlled-fault candidates, exposing deterministic verifier diagnostics to an LLM repair workflow produced a statistically detectable and substantial increase in verifier+simulator-validated configuration success (approx. +20.51 percentage points; $p = 0.021484375$; 95% CI [0.07692307692307693, 0.358974358974359]). This finding is bounded to the preregistered synthetic task bank, the declared model (gpt-5-mini), and the archived deterministic verifier and simulator. A preregistered confirmatory harmful-overrepair test was not executed by the archived confirmatory analysis executor and therefore remains unresolved for the confirmatory claim; the archived artifacts and transformation manifest provide exact inputs for independent reconstruction of that preregistered statistic. We release the artifact bundle for reproducible reaggregation and recommend multi-model, multi-verifier replications and larger-scale cost/latency studies to assess operational feasibility and safety at NetOps scale.

DISCLOSURE STATEMENT

Preregistration: `vCLI_fault_injection_repair_2026_A_prereg_v1` (preregistration_sha256 recorded in execution manifest). Adapter: `hosted_netops_validator_feedback_repair_v2`. Declared model used for confirmatory execution: `gpt-5-mini` (declared_model_version recorded in execution/model_configuration.json). Exact initial master prompt archived at `provenance/master_prompt.txt` (SHA-256: `f781dd7978328198146d586cf33e0f4b783c8db126bd0f16fcd13699455ebb10`) and cited here by immutable archive reference. Human role: a Research Director selected the topic family and pre-lock constraints; after the autonomy lock the autonomous system performed literature verification, preregistration, experiment execution, analysis, and manuscript generation; no manual human editing of technical manuscript content occurred after the lock. Execution semantics: execution manifest records `temperature = null/unset` in `execution/model_configuration.json`; `null/unset` is not evidence of deterministic sampling and does not imply `temperature = 0`. Pairing semantics: exactly one sampled initial generation per task pair was performed and reused by both arms under `shared_controlled_fault_candidate` semantics; baseline denotes blind repair (no validator diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. Harmful-change mapping and reconstructability: the preregistered harmful-change rules were frozen and the transformation artifact implementing the mapping is archived (see `execution/transformation_manifest.json` and the full artifact manifest `execution/execution_manifest.json` in the evidence bundle). Per-episode raw traces and scoring artifacts required to reproduce all aggregated confirmatory and preregistered secondary statistics are archived (`execution/raw_results.jsonl`; `execution/scoring/*.json`; `analysis/*.csv`). The study followed

the preregistered stopping rule; no silent adapter substitution occurred.

REFERENCES

- [1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [2] <https://arxiv.org/abs/2607.20478>
- [3] Arnav Gupta, "Verifier Exploitation in NLI-Guided Iterative Refinement: A Controlled Empirical Analysis", 2026, 10.21203/rs.3.rs-10187492/v1
- [4] <http://arxiv.org/abs/2603.19328>
- [5] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729